

Reverse Engineering CTF Writeup

Challenge: Recover IP address and port from client.exe (flag format: {ipaddress_port})

Tools Used

- strings
- file
- Decompiler explorer
- Ghidra (or IDA Free)

Step 1 - Identify the executable

Open a terminal and determine the file type.

file client.exe

Result: PE32+ (64-bit) Windows executable. This tells us we should use a Windows reverse engineering tool such as Ghidra.

```
(h4kim@kali)-[~/Downloads]
$ file client.exe
client.exe: PE32+ executable (console) x86-64 (stripped to external PDB), for MS Windows, 10 sections
```

Step 2 - Extract printable strings

strings client.exe > strings.txt

grep -i -E "192|socket|connect|winsock|cmd" strings.txt

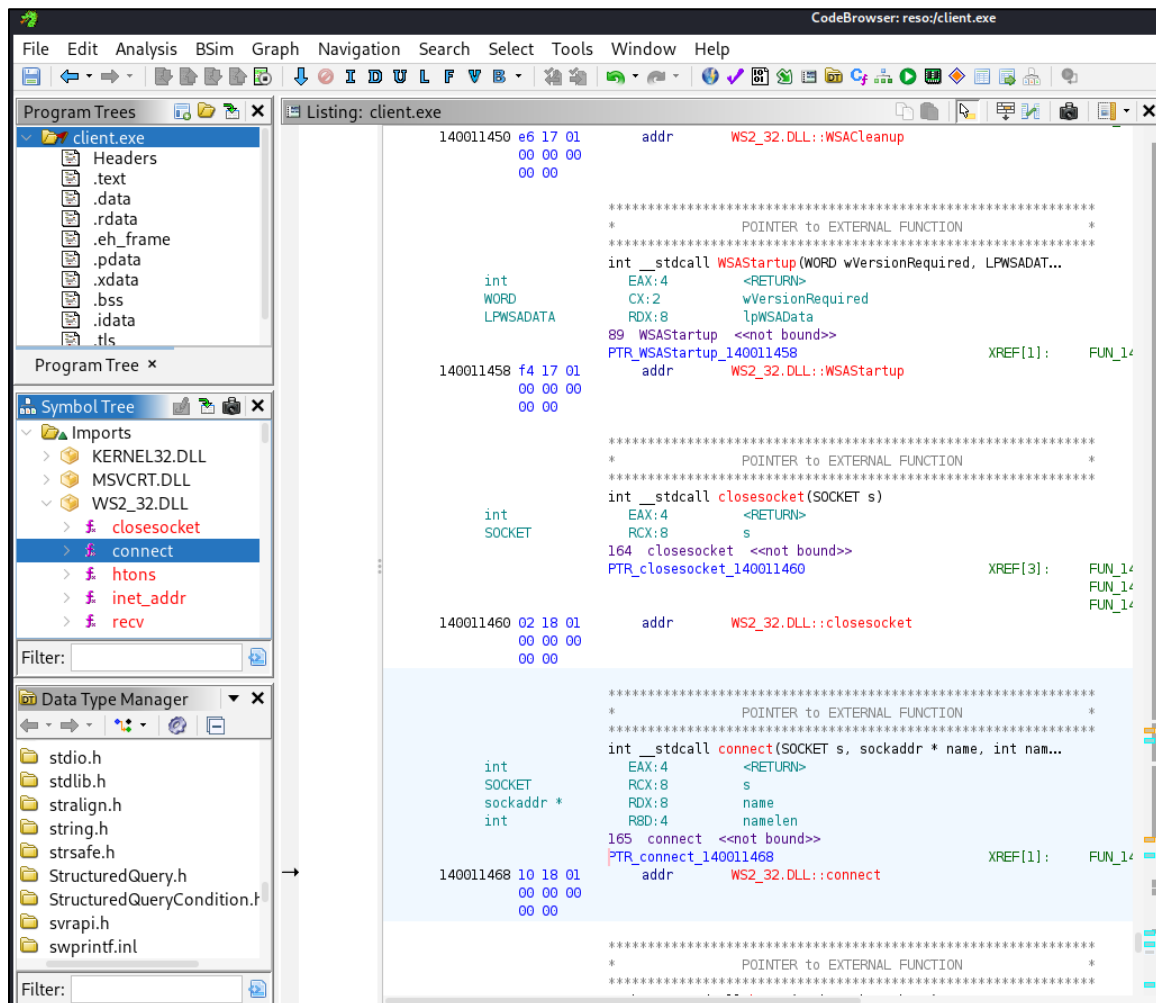
Interesting strings include Winsock error messages and part of an IP address. This suggests the binary communicates over the network.

```
(h4kim@kali)-[~/Downloads]
$ strings client.exe | grep -i -E "192|socket|connect|winsock|cmd"
192.168.H
Error: WinSock Initialization Failed
Error: Socket Creation Failed
Error: Connection Failed
cmd.exe /c %s
closesocket
connect
socket
```

Step 3 - Open in Ghidra

1. Create a new project.
2. Import client.exe.
3. Allow Auto Analysis.
4. Open Symbol Tree -> Imports.
5. Look for WSStartup, socket, connect, inet_addr, htons.

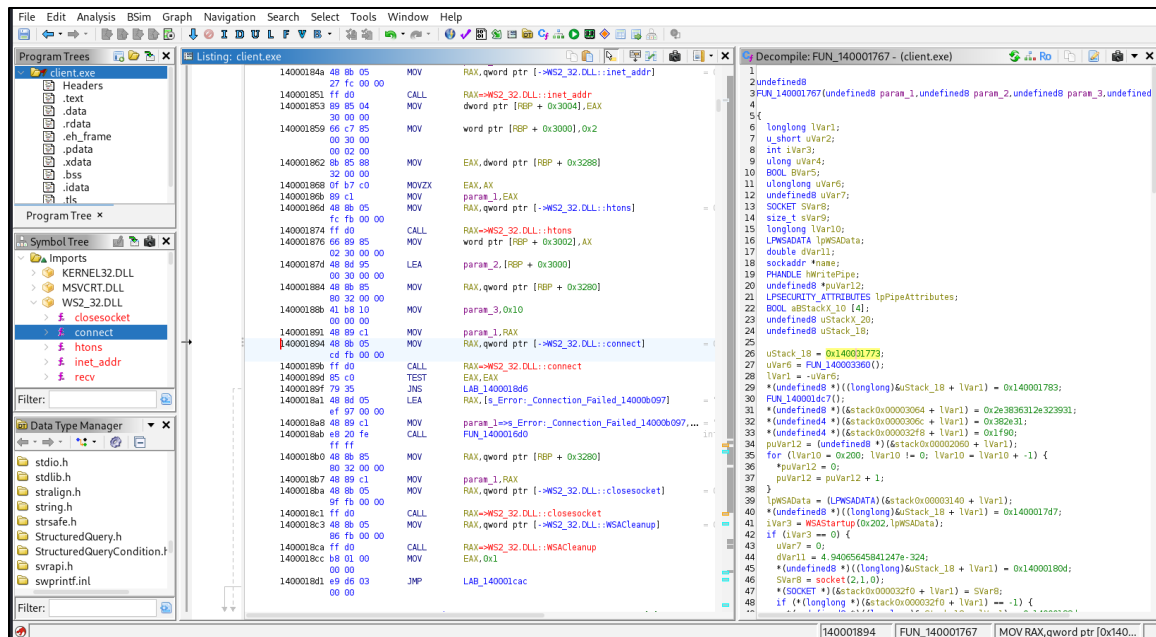
These APIs are commonly used by Windows networking applications. Following them quickly leads to the connection routine.



Step 4 - Locate the connection function

Double-click the imported function 'connect'. Press X (Show References) to find where connect() is called. Navigate to the caller.

Nearby you should see code preparing a sockaddr_in structure, setting an IP address and port before calling connect().



Step 5 - Recover the IP address

movabs rax, 0x2e3836312e323931

The constant is stored as ASCII bytes.

31 39 32 2E 31 36 38 2E

ASCII -> 192.168.

Another constant in the function is:

0x382e31

ASCII -> 1.8

Combining both parts gives the complete IP address:

192.168.1.8



Step 6 - Recover the port

mov DWORD PTR [...], 0x1F90

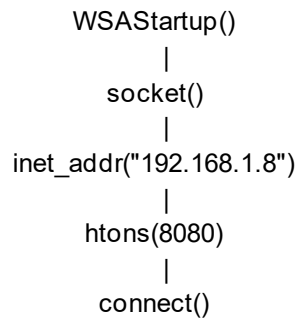
The value 0x1F90 is passed to htons(), which converts the port to network byte order.

0x1F90 (hex) = 8080 (decimal)

Therefore, the destination port is 8080.

14000179e	c7 85 88	MOV	dword ptr [RBP + 0x3288], 0x1f90
	32 00 00		
	90 1f 00 00		

Step 7 - Verify program logic



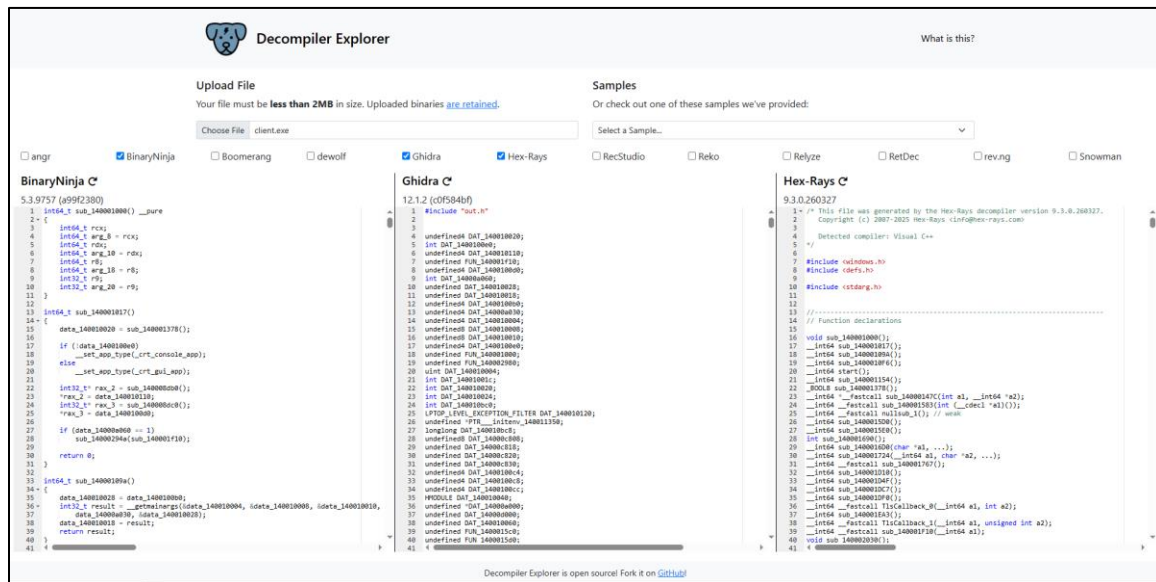
This confirms the executable attempts to connect to 192.168.1.8 on TCP port 8080.

Network clients usually hardcode or compute the server address before calling connect(). By locating Winsock APIs and examining the arguments passed to inet_addr() and htons(), we can recover the server details without executing the program.

Flag: {192.168.1.8_8080}

Shortcut tips:

Use the decompiler explorer



In binary ninja you can found the ip connected:

```
BinaryNinja C
5.3.9757 (a99f2380)
223 }
224 uint64_t sub_140001724(void* arg1, char* arg2, int64_t arg3)
225 {
226     int64_t arg_18 = arg3;
227     int64_t r9;
228     int64_t arg_20 = r9;
229     return sub_140003410(arg1, arg2, &arg_18);
230 }
231
232 int64_t sub_140001767()
233 {
234     sub_140003360(0x3318);
235     sub_140001dc7();
236     int64_t cp;
237     __builtin_strcpy(&cp, "192.168.1.8");
238     uint16_t hostshort = 0x1f90;
239     char var_12b8[0x1004];
240     __builtin_memset(&var_12b8, 0, 0x1000);
241     WSADATA wSADATA;
242     int32_t rax;
243     int64_t rdx_1;
244     rax = WSStartup(0x202, &wSADATA);
245
246     if (rax)
247     {
248         sub_1400016d0("Error: WinSock Initialization Failed\n", rdx_1);
249         return 1;
250     }
251
252     SOCKET s;
253     int64_t rdx_2;
254     s = socket(2, SOCK_STREAM, 0);
255
256     if (s == -1)
257     {
258         sub_1400016d0("Error: Socket Creation Failed\n", rdx_2);
259         WSACleanup();
260         return 1;
261     }
262 }
```

But for the port need to using ghidra to get easier.